

Reporte Práctica 4: Listas Ligadas Simples

Emmanuel Cruz Durán

21 de septiembre de 2026

1. Reporte Técnico y Práctico

a) Principales complicaciones al desarrollar la práctica: El reto principal al programar una lista ligada desde cero es cambiar la mentalidad de usar índices directos (como en los arreglos) a manejar referencias. Si al intentar eliminar un elemento olvidas “puentear” correctamente el nodo anterior con el siguiente, puedes dejar nodos flotando o perder el resto de la lista en memoria. Además, validar correctamente los casos límite (como cuando la lista está vacía, es decir, `longitud == 0`) fue crucial para evitar errores de ejecución en Python.

b) Definición formal de algoritmos: No hubo algoritmos adicionales porque todo lo que necesitaba ya estaba en el esqueleto de la práctica, inclusive el Dunder Method (`__iter__`). Y en general, solo era completar la implementación de los algoritmos que ya estaban en las Notas, nada que no se pudiera hacer desde los mismo métodos que ya se había definido.

a) Listas vs Conjuntos para el torneo: La gran ventaja de las listas es que sí respetan el orden en el que vas metiendo los datos y sí dejan tener elementos repetidos. En el torneo, importa mucho el orden para saber quién pelea contra quién. Si hubiéramos usado conjuntos matemáticos como en la Práctica 3, se habría borrado a cualquier participante que tuviera el mismo nombre y cinta, y todos quedarían completamente desordenados.

b) ¿Es necesario modificar Participante y Cinta? No, no le toqué nada a esas clases. Esas clases solo guardan la información del competidor y sus reglas de combate. A ellas no les importa para nada si las guardas adentro de un arreglo, un conjunto o una lista ligada. Su lógica interna sigue funcionando exactamente igual sin importar la colección.

c) ¿Qué estructura de datos es mejor: listas, arreglos o conjuntos? Depende mucho de lo que el programa necesite hacer. Si vas a buscar datos constantemente por su posición, el arreglo es mejor porque su acceso es instantáneo. Si necesitas hacer operaciones matemáticas y garantizar que no haya elementos repetidos, los conjuntos son la opción correcta. Pero si vas a estar metiendo y sacando muchísimos datos sin saber cuántos van a ser en total, la lista ligada es la mejor porque su memoria es dinámica y no desperdicia espacio vacío.

d) ¿Qué pasaría si se agregan elementos al final y qué modificaciones harías? Si intentamos agregar datos al final de la lista con el código actual, la función sería lentísima. Por cada dato nuevo tendríamos que recorrer toda la lista desde la cabeza solo para encontrar el último nodo. Para arreglar eso sin dañar el rendimiento, tendríamos que agregar una variable nueva a la clase (como un `self.__cola`) que guarde en todo momento la referencia al último nodo agregado. Así podríamos enganchar el nuevo elemento directamente.

Respuestas de la IA

- a) Las principales ventajas de utilizar listas ligadas simples en comparación con los conjuntos son que permiten mantener un orden específico de los participantes y facilitan la inserción y eliminación de elementos. Además, no es necesario reservar un tamaño fijo de memoria desde el inicio, ya que los nodos se crean dinámicamente conforme se necesitan. Para el problema del torneo, esto permite administrar de manera flexible a los participantes y recorrerlos en el orden en que fueron agregados.
- b) No es necesario modificar las clases `Participante` y `Cinta` de la Práctica 2. La implementación de la lista ligada puede utilizar objetos de estas clases como elementos de sus nodos. Solamente sería necesario crear las clases o estructuras correspondientes a los nodos y a la lista, manteniendo las clases `Participante` y `Cinta` con su funcionamiento original.
- c) No existe una estructura que sea mejor en todos los casos; la elección depende del problema. Para el problema del torneo, las listas ligadas son convenientes cuando se requiere insertar o eliminar participantes con frecuencia y no se conoce de antemano la cantidad de elementos. Los arreglos son más adecuados cuando se necesita acceder directamente a cualquier posición mediante un índice, ya que su acceso es más rápido. Por otro lado, los conjuntos son útiles cuando se necesita evitar elementos repetidos y realizar operaciones de pertenencia de manera eficiente. Por lo tanto, para este problema, las listas pueden resultar más adecuadas si las operaciones principales son inserciones, eliminaciones y recorridos secuenciales.
- d) Si los elementos se agregaran al final de la lista en lugar del inicio, sería necesario recorrer la lista desde el primer nodo hasta encontrar el último para realizar cada inserción. Esto tendría un costo de tiempo de $O(n)$ por inserción. Para hacerlo más eficiente, se podría agregar un atributo `ultimo` o `tail` que mantenga una referencia al último nodo de la lista. De esta manera, cada nuevo elemento podría agregarse directamente después del último nodo y actualizar la referencia `ultimo`. La implementación tendría que modificar el método de inserción y, además, actualizar tanto `ultimo` como `primero` cuando se agregue el primer elemento a una lista vacía.

Análisis Crítico

Comparando mis respuestas con las de la IA. Yo me enfoqué más en cómo afecta la lógica al problema real de la práctica, y la IA dio respuestas más de libro de texto.

Por ejemplo, en la primera pregunta, yo me fui por el hecho de que un conjunto nos borraría a los participantes repetidos (misma cinta, mismo nombre) y arruinaría el torneo. La IA, en cambio, se enfocó más en las ventajas de la memoria dinámica. En la segunda, los dos concordamos en que no hay que tocar las clases `Participante` y `Cinta`; mi razonamiento fue que a la clase "no le importa" dónde la guardes, y la IA lo explicó diciendo que el nodo simplemente envuelve al objeto sin alterar su código.

En la de elegir la mejor estructura, literal respondimos lo mismo: "depende".

Finalmente, en la de agregar elementos al final, los dos notamos que recorrer todo haría la función lentísima (la IA sacó la notación $\mathcal{O}(n)$) y propusimos exactamente la misma solución: usar una variable extra como `cola` o `tail`. Solo le doy el punto a la IA de que se acordó de mencionar el caso especial de actualizar los apuntadores cuando la lista está vacía.